Week 15 In-Class RTL Exercise

Fair Shared-Accelerator Arbitration

Purpose: This handout gives exact steps for running the Docker environment, observing the baseline behavior, implementing the required RTL change, and validating the result during class. The exercise is scoped for a two-hour in-class session, not a homework assignment.

Item	Student-facing value
Main editable file	rtl/shared_accel_arbiter.sv
Required task	Implement ARB_POLICY == 1 round-robin arbitration
Do not edit	tb/tb_shared_accel_arbiter.sv, scripts, Makefile, and fixed accelerator lane unless instructed by the instructor
Primary Docker image	amansinhaatnycu/osp-week14-security-exercise:latest
Fallback behavior	If the primary image cannot be pulled, docker/run_student_container.sh builds a local fallback image
Final pass command	make check
Expected starter behavior	Fixed priority passes correctness; placeholder round-robin fails fairness
Expected completed behavior	Round-robin passes correctness and fairness

Contents

1. Learning goals and time plan

- **Functional correctness:** Confirm that both requesters complete all issued commands without duplicate, missing, or incorrect responses.
- **Fairness measurement:** Use max_wait0 and max_wait1 as observable RTL-level evidence of waiting-time imbalance.
- **Policy change:** Replace a fixed-priority placeholder with a small round-robin policy that improves worst-case waiting time.

- **Tool workflow:** Use Docker, Verilator, Python log parsing, and Yosys in a repeatable open-source flow.
- **Design contract:** Preserve the valid/ready interface and testbench-visible behavior while changing only the arbitration decision.

Class time	Activity	Expected output
0-15 min	Start Docker container and run environment check	make env-check passes
15-35 min	Run fixed-priority and starter round-robin simulations	Correctness passes; starter round-robin fails fairness
35-55 min	Read the RTL and identify the student TODO region	Students understand request, grant, and last_grant signals
55-90 min	Implement round-robin in ARB_POLICY == 1 branch	RTL compiles without editing the testbench
90-110 min	Run validation and compare metrics	make check passes after the correct fix
110-120 min	Discuss results and optionally inspect waveform or Yosys report	Students can explain why fairness improved

2. What the hardware model represents

The exercise models a common prototype-system problem: two clients share one accelerator lane. A real system could map these clients to two CPU cores, two DMA engines, two software queues, or two tenants. The accelerator lane can process only one command at a time, so an arbiter must decide which requester receives service when both are active.

- **Requester 0 and requester 1:** Each requester continuously issues tagged commands until it has sent the configured number of commands.
- **Shared accelerator lane:** The lane accepts one command through a valid/ready interface and returns one deterministic response after a fixed latency.
- **Arbiter role:** The arbiter selects either requester 0 or requester 1 and forwards the chosen tag, opcode, and length to the lane.
- **Fixed-priority baseline:** Requester 0 wins whenever requester 0 is valid, so requester 1 can wait much longer under contention.
- **Round-robin goal:** When both requesters are valid, the requester that did not win last time should receive service next.
- **Measurement goal:** A better policy should keep the maximum wait of both requesters below the configured fairness limit for the default workload.

Key idea: The baseline design is not broken in the usual correctness sense. It completes all commands. The problem is that correctness alone does not guarantee fair service for shared hardware resources.

3. File package and what each file does

Path	Purpose	Student action
rtl/shared_accel_arbiter.sv	Contains the arbitration policy and wait-time counters.	Edit only the ARB_POLICY == 1 branch for the required task.

rtl/shared_accel_lane.sv	Models a fixed-latency accelerator datapath.	Do not edit for the required task.
rtl/shared_accel_top.sv	Connects the arbiter to the accelerator lane.	Do not edit for the required task.
tb/tb_shared_accel_arbiter.sv	Self-checking SystemVerilog testbench.	Do not edit; it defines the contract.
Makefile	Builds and runs simulations, comparisons, waveforms, and Yosys synthesis.	Use the provided targets; do not modify during class.
scripts/parse_sim_log.py	Extracts METRIC lines into CSV files.	Do not edit.
scripts/compare_metrics.py	Compares fixed-priority and round-robin CSV metrics.	Do not edit.
docker/run_student_container.sh	Starts the intended Docker environment and mounts the exercise folder.	Run from the host terminal.
docker/verify_env.sh	Checks make, python3, Verilator, and Yosys inside the container.	Called by make env-check.
reports/, logs/, results/, waves/	Generated output directories.	Inspect as needed; safe to delete with make clean.

4. Docker setup: exact host commands

Run this section on your host machine, not inside Docker yet. The commands assume you received the student starter ZIP file from the instructor.

4.1 macOS or Linux host

1. **Open a terminal:** Use Terminal, iTerm, or another shell on the machine where Docker is installed.
2. **Go to the download location:** Replace the path below with the folder where the ZIP file was saved.

```
cd ~/Downloads
```

3. **Unzip the starter package:** This creates a folder named week15_student.

```
unzip week15_starter_files.zip
cd week15_starter_files
```

4. **Make Docker scripts executable:** This avoids permission errors on macOS and Linux.

```
chmod +x docker/*.sh
```

5. **Start the container:** The script mounts the current exercise folder into /workspace inside the container.

```
./docker/run_student_container.sh
```

4.2 Windows host using WSL2

- **Recommended setup:** Use Docker Desktop with WSL2 integration enabled and run the commands from an Ubuntu WSL terminal.
- **Path convention:** A Windows folder such as C:\Users\YourName\Downloads appears inside WSL as /mnt/c/Users/YourName/Downloads.

```
cd /mnt/c/Users/<YourName>/Downloads
unzip week15_starter_files.zip
cd week15_starter_files
chmod +x docker/*.sh
./docker/run_student_container.sh
```

4.3 What the Docker script does

- **Primary image:** The script first tries to pull amansinhaatnycu/osp-week14-security-exercise:latest, which already contains the open-source tools needed for this RTL lab.
- **Fallback image:** If the pull fails, the script builds a local fallback image from docker/Dockerfile.fallback using apt packages for Verilator, Yosys, GTKWave, make, and Python.
- **Mounted workspace:** The exercise folder on the host is mounted as /workspace in the container, so edits made on the host or inside the container affect the same files.
- **Container prompt:** After the script succeeds, you should be inside a shell whose working directory is /workspace.

Important: Do not copy files manually into the container. The folder is mounted. Edit the files in week15_student on your host editor or inside the container, then run make commands from /workspace.

5. First commands inside the container

After docker/run_student_container.sh finishes, run the following commands inside the container.

```
pwd
ls
make help
```

- **Expected pwd:** The command should print /workspace.
- **Expected files:** You should see Makefile, rtl, tb, scripts, docker, docs, and yosys directories.
- **Help target:** make help prints the supported simulation, checking, comparison, waveform, synthesis, and cleanup commands.

5.1 Environment check

```
make env-check
```

- **Tool check:** The script verifies make, python3, Verilator, and Yosys.
- **Sanity simulation:** It runs a small fixed-priority simulation with N0=4 and N1=4.
- **Expected result:** You should see PASS correctness, a fairness-disabled note, a CSV write message, and Environment check passed.
- **Not a solution test:** env-check only confirms that the tools and package run; it does not prove that the round-robin TODO is completed.

6. Baseline measurements before editing RTL

Before editing anything, collect the baseline outputs. This helps you understand the difference between functional correctness and fairness.

6.1 Run the fixed-priority baseline

```
make sim POLICY=fixed
```

- **Expected policy field:** The log should contain METRIC policy=0.
- **Expected correctness:** The log should contain PASS correctness because all commands complete with the expected results.
- **Expected fairness behavior:** The log should say fairness check disabled and max_wait1 should be much larger than max_wait0 for the default workload.

- **Typical default pattern:** You may see max_wait0 near 4 and max_wait1 near 160, showing strong priority bias toward requester 0.

6.2 Run the starter round-robin placeholder

```
make sim POLICY=rr CHECK_FAIR=1
```

- **Expected policy field:** The log should contain METRIC policy=1.
- **Expected starter behavior:** Before you edit the RTL, this should still pass correctness but print FAIL fairness.
- **Reason:** The ARB_POLICY == 1 branch in the starter file deliberately behaves like fixed priority so that you can observe and fix the issue.
- **Classroom-friendly failure:** The updated testbench reports FAIL fairness without aborting Verilator; make check is the strict pass/fail target.

6.3 Compare the two policies before the fix

```
make compare
```

- **CSV files:** The command creates results/fixed_metrics.csv and results/rr_metrics.csv.
- **Expected before fix:** The fixed and rr metrics should look very similar because the rr branch is still a placeholder.
- **What to notice:** The completed command counts may be correct while max_wait values reveal unfair service.

Command	Expected before RTL edit	Meaning
make env-check	Passes	Tools and package are usable
make sim POLICY=fixed	PASS correctness; fairness disabled	Fixed priority is correct but unfair
make sim POLICY=rr CHECK_FAIR=1	PASS correctness; FAIL fairness	Round-robin branch is still the placeholder
make check	Fails before the fix	Strict final check correctly detects the unfinished TODO

7. Required RTL implementation task

Required edit: Open rtl/shared_accel_arbiter.sv and complete only the ARB_POLICY == 1 branch. Do not change the testbench, parser scripts, Makefile, top module, or accelerator lane for the required task.

7.1 Locate the TODO region

```
grep -n "STUDENT TODO\|ARB_POLICY == 1" rtl/shared_accel_arbiter.sv
```

- **Target branch:** Find the always_comb block and the else if (ARB_POLICY == 1) branch.
- **Current placeholder:** The starter code grants requester 0 whenever req0_valid is high, which repeats the fixed-priority behavior.
- **Required replacement:** Change only this branch so that it implements fair round-robin selection.

7.2 Understand the signals before editing

Signal	Direction or role	Meaning
req0_valid, req1_valid	Inputs to arbiter	A requester has a command waiting.

req0_ready, req1_ready	Outputs from arbiter	The selected requester may transfer its command when ready is high.
grant0_sel, grant1_sel	Internal combinational choices	Exactly one should be high when a valid requester is selected.
cmd_valid	Output to lane	The arbiter has selected a command for the accelerator lane.
cmd_ready	Input from lane	The lane can accept a command this cycle.
cmd_valid && cmd_ready	Handshake event	A command is actually accepted by the lane.
last_grant	Internal state	Records which requester last received an accepted command.
grant_count0, grant_count1	Metrics	Counts accepted grants for each requester.
max_wait0, max_wait1	Metrics	Tracks the longest continuous waiting period for each requester.

7.3 Required round-robin behavior

Implement the following behavior in the ARB_POLICY == 1 branch:

- **No request valid:** Grant neither requester.
- **Only requester 0 valid:** Grant requester 0.
- **Only requester 1 valid:** Grant requester 1.
- **Both requesters valid:** Grant the requester that was not granted last time.
- **Last grant value:** If last_grant records requester 0 as the previous accepted winner, choose requester 1 during a tie; if it records requester 1, choose requester 0.
- **State update location:** Do not add a new state update unless necessary; the provided sequential block already updates last_grant when cmd_ready and a grant selection are true.
- **Mutual exclusion:** Never set grant0_sel and grant1_sel high in the same cycle.

7.4 What not to do

- **Do not edit the testbench:** Changing the testbench hides bugs instead of fixing the arbiter.
- **Do not change FAIR_MAX_WAIT:** The default limit is part of the exercise contract.
- **Do not force max_wait values:** The metrics must come from the actual arbitration behavior.
- **Do not always alternate blindly:** When only one requester is valid, the arbiter should grant that requester instead of waiting for the other requester.
- **Do not update last_grant on an unaccepted command:** A command should affect last_grant only when the lane accepts it through the valid/ready handshake.
- **Do not modify cmd mux assignments:** The existing muxes already forward the selected requester correctly when grant signals are correct.

7.5 Editing options

- **Host editor option:** Because the folder is mounted into Docker, you can edit rtl/shared_accel_arbiter.sv using VS Code, Sublime Text, Vim, or another editor on the host.
- **Container editor option:** If nano or vim is available inside the container, you can edit directly from /workspace.

```
nano rtl/shared_accel_arbiter.sv
# or
vim rtl/shared_accel_arbiter.sv
```

8. Validation after the fix

After editing rtl/shared_accel_arbiter.sv, run the commands below. Use make clean first if you are unsure whether generated files came from an earlier build.

```
make clean
make sim POLICY=rr CHECK_FAIR=1
```

- **Expected correctness:** The log must contain PASS correctness.
- **Expected fairness:** The log must contain PASS fairness max_wait_limit=32.
- **Expected wait metrics:** Both max_wait0 and max_wait1 should be at or below FAIR_MAX_WAIT for the default workload.
- **Expected grants:** For N0=32 and N1=32, grants0 and grants1 should both be 32.

8.1 Run the strict final check

```
make check
```

- **Before the fix:** This command should fail because it sees FAIL fairness in logs/rr_sim.log.
- **After the fix:** This command should pass and print PASS: Correctness and fairness checks passed.
- **Why check is strict:** It treats fairness failure as a failed exercise, even though make sim itself is classroom-friendly and continues after printing metrics.

8.2 Compare metrics after the fix

```
make compare
```

- **Look at max_wait:** The round-robin max_wait values should be much more balanced than the fixed-priority values.
- **Look at total completion:** Both policies should complete all requests; the difference is service fairness, not basic functionality.
- **Use the CSV files:** results/fixed_metrics.csv and results/rr_metrics.csv can be opened as plain text or imported into a spreadsheet.

9. Optional waveform and Yosys exploration

These commands are useful if you finish early or if the instructor asks the class to inspect the design more deeply.

9.1 Generate a waveform

```
make wave POLICY=rr CHECK_FAIR=1 TRACE=1
```

- **Waveform path:** The VCD file should be written under waves/shared_accel_arbiter.vcd.
- **Open waveform:** If GTKWave is available through your display setup, use gtkwave waves/shared_accel_arbiter.vcd.

```
gtkwave waves/shared_accel_arbiter.vcd
```

- **Signals to inspect:** Start with req0_valid, req1_valid, req0_ready, req1_ready, cmd_valid, cmd_ready, cmd_src, grant_count0, grant_count1, max_wait0, and max_wait1.
- **Expected pattern:** When both requesters remain valid, cmd_src should alternate between requester 0 and requester 1 after the round-robin fix.

9.2 Run Yosys synthesis estimates

```
make yosys POLICY=fixed
make yosys POLICY=rr
```

- **Report files:** Yosys writes reports/fixed_area.txt and reports/rr_area.txt.
- **What to compare:** Look for the number of cells and the presence of simple mux/comparison logic.
- **Expected trade-off:** Round-robin should add a small amount of control logic while significantly improving worst-case waiting time.
- **Class discussion:** This is a typical hardware/software co-design trade-off: a few extra gates can improve service quality for shared resources.

10. What to submit before leaving class

- **Modified RTL file:** Submit or show rtl/shared_accel_arbiter.sv with the completed ARB_POLICY == 1 branch.
- **Terminal output:** Submit copied output or a screenshot showing make sim POLICY=fixed, make sim POLICY=rr CHECK_FAIR=1, and make check.
- **Metric explanation:** Write one or two sentences explaining how max_wait0 and max_wait1 changed after the round-robin fix.
- **No homework expectation:** The required part should be completed during class. The optional aging policy and deeper waveform analysis are enrichment tasks, not required unless assigned separately.

11. Troubleshooting guide

Symptom	Likely cause	What to do
docker: command not found	Docker is not installed or not on PATH.	Install/start Docker Desktop or use the instructor-provided lab machine.
Cannot connect to Docker daemon	Docker service is not running.	Start Docker Desktop or ask the TA to start the Docker daemon.
permission denied: docker/*.sh	Scripts are not executable.	Run chmod +x docker/*.sh from the week15_student folder.
Primary image pull fails	Network issue or DockerHub access issue.	The run script should build the fallback image automatically. If the build also fails, ask the instructor for a prebuilt image.
make env-check says missing verilator or yosys	The container is not the intended image or fallback build failed.	Exit the container and rerun ./docker/run_student_container.sh.
Verilator width warnings stop build	Old package or manually edited testbench introduced width mismatches.	Use the updated package and do not edit tb/tb_shared_accel_arbiter.sv.
make sim POLICY=rr CHECK_FAIR=1 still prints FAIL fairness	Round-robin branch still behaves like fixed priority or grants wrong requester on ties.	Recheck the both-valid case and last_grant tie-breaking.
FAIL correctness	The arbiter is dropping, duplicating, or corrupting commands.	Check that only one grant is selected and that you did not modify cmd muxes or tags.
FAIL timeout	The arbiter stopped granting or left both grant signals low when requests were valid.	Check all valid cases: only req0, only req1, and both valid.

make check fails but make sim seemed okay	make sim reports metrics; make check enforces fairness strictly.	Open logs/rr_sim.log and search for FAIL fairness or FAIL correctness.
No waveform file	TRACE was not enabled or simulation did not reach waveform dump.	Run make wave POLICY=rr CHECK_FAIR=1 TRACE=1.
GTKWave does not open on remote server	No GUI forwarding or X server.	Use the VCD file later on a local machine or ask the instructor to project a waveform.

12. Reasonable hints

Use these hints gradually: Try Hint 1 first. Move to later hints only if you are stuck. The goal is to understand the design, not to blindly paste code.

- **Hint 1:** The accelerator lane is not the problem. The fixed-priority arbiter creates the unfair waiting time.
- **Hint 2:** The required change belongs in the combinational selection logic inside the ARB_POLICY == 1 branch.
- **Hint 3:** The single-request cases are simple: if only requester 0 is valid, grant requester 0; if only requester 1 is valid, grant requester 1.
- **Hint 4:** The both-requesters-valid case is the only case that needs the round-robin decision.
- **Hint 5:** Treat last_grant as the identity of the most recent accepted winner. During a tie, choose the other requester.
- **Hint 6:** Do not change last_grant in always_comb. It is a state variable updated in the sequential block when the lane accepts a command.
- **Hint 7:** If fairness still fails, inspect logs/rr_sim.log and check whether max_wait1 remains close to the fixed-priority value.
- **Hint 8:** If correctness fails, your grant logic may be selecting both requesters, selecting an invalid requester, or preventing a valid requester from ever being served.

13. Concept check questions

Q1. Why can a design pass correctness while still being unsuitable for a shared prototype system?

Q2. Which metric in this exercise exposes unfair service, and why is it more informative than completed command count alone?

Q3. Why should last_grant update only when a command is accepted by the accelerator lane?

Q4. What could go wrong if the arbiter alternates even when only one requester is valid?

Q5. Why is modifying the testbench an invalid way to pass this exercise?

Q6. What does Yosys help you discuss after the fairness fix?

Appendix A: Command cheat sheet

```
# Host: unpack and start container
unzip week15_starter_files.zip
cd week15_starter_files
chmod +x docker/*.sh
./docker/run_student_container.sh

# Inside container: inspect and check environment
pwd
ls
make help
make env-check
```

```
# Baseline observations before editing
make sim POLICY=fixed
make sim POLICY=rr CHECK_FAIR=1
make compare

# Edit required file
# rtl/shared_accel_arbiter.sv

# Validate after editing
make clean
make sim POLICY=rr CHECK_FAIR=1
make check
make compare

# Optional
make wave POLICY=rr CHECK_FAIR=1 TRACE=1
make yosys POLICY=fixed
make yosys POLICY=rr

# Cleanup
make clean
```

Appendix B: Expected log patterns

Stage	Command	Expected key lines
Environment sanity	make env-check	PASS correctness; NOTE fairness check disabled; Environment check passed
Fixed baseline	make sim POLICY=fixed	METRIC policy=0; PASS correctness; max_wait1 much larger than max_wait0
Starter rr before fix	make sim POLICY=rr CHECK_FAIR=1	METRIC policy=1; PASS correctness; FAIL fairness
Completed rr after fix	make sim POLICY=rr CHECK_FAIR=1	METRIC policy=1; PASS correctness; PASS fairness
Final check	make check	PASS: Correctness and fairness checks passed